

面向数据智能协同的翼型工程数据库系统设计与实现

一、项目背景

翼型是航空航天、无人机、流体机械等工程系统中的基础对象。对于一个翼型，既有几何形状数据，也有在不同工况下的性能数据，例如攻角、雷诺数条件下的升力系数、阻力系数、升阻比等。随着工程数据规模扩大，翼型数据管理不再只是文件存储问题，而逐渐成为数据库系统问题：如何对多源数据进行统一建模，如何支持高效查询与统计分析，如何维护数据版本，如何发现异常数据，如何让数据库系统与大模型形成可靠协同，而不是简单拼接。

本项目以翼型工程数据为对象，设计并实现一个以数据库为核心的数据管理与分析系统。系统不仅要支持翼型几何数据、性能数据和版本数据的组织与查询，还要体现数据库设计、完整性约束、索引优化、事务处理、数据治理，以及 AI 协同条件下的 SQL 审计与结果解释能力。

二、项目目标

理解翼型工程数据的结构特点，并建立合理的数据模型；能够将多源数据、版本信息和分析需求映射为规范化的数据库模式；能够使用主键、外键、唯一约束、检查约束等机制保证数据一致性；能够针对核心查询设计索引并分析执行计划；能够实现基本的数据治理机制，包括异常数据识别、版本追踪和来源说明；能够审查自然语言生成 SQL 的正确性，而不是盲信 AI 输出；能够对数据库返回结果进行智能解释，并识别大模型在工程语境中的“合理但错误”现象；能够对数据库与大模型协同的边界形成较为清晰的认识。

三、项目总体任务

每个小组完成一个可运行的“翼型工程数据库系统”。系统围绕以下四个核心任务：

任务 1：工程数据管理。系统需要管理翼型基本信息、几何坐标、性能记录、数据版本、数据来源等信息，并能够完成录入、查询、更新、标记失效等操作。

任务 2：数据库优化与机制实现。系统需要体现规范化设计、完整性约束、索引设计、执行计划分析，以及至少一个事务场景或并发一致性场景。

任务 3：数据治理与质量控制。系统需要能够发现明显异常数据，并保留数据版本与来源信息，使数据具有可解释性与可追踪性。

任务 4：数据智能协同。系统需要实现自然语言问题到 SQL 的受控生成与审计，并对数据库查询结果进行智能解释，同时分析大模型输出的正确性边界。

四、数据来源与规模要求

4.1 数据源说明

本项目的数据来源分为两类：一类是真实公开工程数据，另一类是基于真实数据进行合理扩增后形成的课程实验数据。两类数据应在系统中明确区分，不得混用而不加说明。

本项目建议采用 **UIUC Airfoil Coordinates Database** 作为翼型几何数据的基础来源。该数据库由 UIUC Applied Aerodynamics Group 维护，当前公开提供约 1,650 个翼型的坐标文件。其数据通常以 .dat 文本文件形式提供，文件中可能包含以 # 开头的注释行，且注释行可能出现在文件任意位置，因此在编写解析程序时必须正确处理注释、空行和标题行。数据库页面还说明，这些坐标通常按照从上表面尾缘出发、经过前缘、再回到下表面尾缘的顺序组织，这一点在数据导入和表面分段处理中需要特别注意。

除几何数据外，本项目的性能数据可以采用两种方式获得。第一种方式是使用公开可获取的翼型性能实验数据或低雷诺数翼型数据子集；第二种方式是基于已有翼型几何数据，通过程序生成满足基本物理趋势的课程实验性能数据。允许使用“真实几何数据 + 可控生成性能数据”的混合方案。UIUC 相关低速翼型实验数据页面说明，其部分低雷诺数翼型实验数据可在线获取为 ASCII 文本。

在课程项目中，不用导入全部公开翼型数据，可选取 60 至 100 个翼型作。

4.2 数据使用原则

本项目中的真实数据与生成数据必须遵循可追踪、可区分、可解释的原则。各小组在导入和构建数据库时，应在表结构或元数据中显式保留数据来源信息，例如标记某条数据来自 UIUC 原始坐标文件、公开实验数据、程序扩增版本或人工注入的异常样本。对于性能数据，若采用程序生成方式，必须在报告中说明生成规则、参数设定及其合理性边界，不能将纯随机生成结果伪装为真实工程测量数据。

所有生成数据仅用于数据库课程实验，不应被表述为真实高保真气动仿真结果。课程允许使用简化的物理趋势模型生成性能数据，目的在于支撑数据库建模、索引优化、事务实验、数据治理和 AI 协同测试，不是替代 CFD（计算流体力学 Computational Fluid Dynamic）仿真或风洞实验。

4.3 数据规模建议

为保证项目具有足够的数据量支撑数据库实验，建议各小组构建的数据集至少满足以下规模要求：

翼型数量不少于 60 个；

每个翼型至少包含 80 个以上几何坐标点；

性能记录总数不少于 3000 条；

性能数据至少覆盖多个攻角条件以及 2 个及以上雷诺数条件；

数据集中应包含少量异常数据或可疑数据，用于支持数据质量治理实验。

对于希望达到更高完成度的小组，可以进一步扩展到 80 个以上翼型、每翼型 120 个以上坐标点、总性能记录超过 5000 条，以支持更充分的索引实验、分区实验和 AI 协同测试。

4.4 示例数据生成的总体思路

说明书提供的示例数据生成脚本仅作为参考框架，不是最终答案。各小组可以在此基础上修改、扩展和优化，但必须说明自己的设计选择。

示例数据生成遵循以下基本思路：首先从 UIUC 翼型坐标数据库中读取部分真实翼型 .dat 文件，构建基础翼型及其几何坐标数据；然后在真实几何基础上通过轻微参数扰动生成若干版本变体，以模拟工程设计中翼型版本演化的情形；接着基于简化的性能趋势模型，在不同攻角和雷诺数条件下生成性能数据，并加入适度随机扰动，模拟实验或仿真中可能出现的测量波动；最后以极低概率注入异常数据，用于后续数据治理和异常检测实验。

这种做法的目的不是生成高保真的航空仿真数据库，而是在保证一定工程语义的前提下，为课程项目中的模式设计、约束控制、查询分析、性能优化、数据治理和 AI 协同审计提供可用的数据基础。

4.5 示例生成脚本的输出内容

建议示例脚本至少输出以下几类数据文件：

airfoils.csv：翼型基础信息表，用于存储翼型编号、名称、来源、家族类别及是否为生成数据等信息；

coordinates.csv：坐标点数据表，用于存储翼型在特定版本下的逐点坐标数据；

performance.csv：性能数据表，用于存储翼型在不同攻角、雷诺数等工况下的升力系数、阻力系数等性能记录；

data_versions.csv：版本信息表，用于存储版本编号、所属翼型、版本号及版本类型；

anomalies.csv 或等价标记字段：用于标记异常数据记录，支撑后续质量治理实验。

在数据库实现时，根据自己的表结构设计对这些 CSV 进行规范化导入，而不是直接把所有字段堆叠到单表中。

4.6 示例数据生成脚本框架

下面给出一个可供课程使用的示例脚本框架。该框架展示了如何从 UIUC .dat 文件读取坐标数据、如何生成版本化翼型数据、如何生成课程实验所需的性能记录，以及如何注入少量异常数据。学生可以在此基础上进行完善。

```
from __future__ import annotations
```

```
import csv

import math

import random

from dataclasses import dataclass

from pathlib import Path

from typing import List, Tuple


RAW_DIR = Path("project_data/raw_uiuc")

OUT_DIR = Path("project_data/output")


OUT_DIR.mkdir(parents=True, exist_ok=True)


@dataclass

class AirfoilMeta:

    airfoil_id: str

    name: str

    source: str

    family: str

    is_generated: bool


@dataclass

class CoordinatePoint:

    airfoil_id: str

    version_id: str

    point_order: int

    x: float

    y: float
```

```
surface: str
```

```
@dataclass
```

```
class PerformanceRecord:
```

```
    airfoil_id: str
```

```
    version_id: str
```

```
    alpha_deg: float
```

```
    reynolds_number: int
```

```
    cl: float
```

```
    cd: float
```

```
    source_type: str
```

```
    is_anomaly: bool
```

```
def read_uiuc_dat(file_path: Path) -> List[Tuple[float, float]]:
```

```
    coords: List[Tuple[float, float]] = []
```

```
    with file_path.open("r", encoding="utf-8", errors="ignore") as f:
```

```
        lines = f.readlines()
```

```
    for line in lines:
```

```
        line = line.strip()
```

```
        if not line or line.startswith("#"):
```

```
            continue
```

```
        parts = line.split()
```

```
        if len(parts) != 2:
```

```
            continue
```

```

try:
    x = float(parts[0])
    y = float(parts[1])
    coords.append((x, y))
except ValueError:
    continue

if len(coords) < 20:
    raise ValueError(f"{file_path.name} 坐标点过少，疑似解析失败")

return coords

```

```

def infer_surface(point_index: int, total_points: int) -> str:
    halfway = total_points // 2
    return "upper" if point_index < halfway else "lower"

```

```

def make_version_id(airfoil_id: str, version_no: int) -> str:
    return f"{airfoil_id}_v{version_no}"

```

```

def perturb_coordinates(
    base_coords: List[Tuple[float, float]],
    thickness_scale: float,
    camber_shift: float,
) -> List[Tuple[float, float]]:
    result: List[Tuple[float, float]] = []
    for x, y in base_coords:
        new_y = y * thickness_scale + camber_shift

```

```
        result.append((x, new_y))

    return result
```

```
def synthetic_aero_model(alpha_deg: float, reynolds_number: int) -> Tuple[float, float]:
```

```
    alpha_rad = math.radians(alpha_deg)

    cl = 2 * math.pi * alpha_rad

    re_factor = (reynolds_number / 1_000_000) ** 0.05

    cl *= re_factor

    if alpha_deg > 12:

        cl *= max(0.6, 1.0 - 0.08 * (alpha_deg - 12))

    cd = 0.008 + 0.01 * (cl ** 2) + 0.00015 * (alpha_deg ** 2)

    cl *= random.normalvariate(1.0, 0.02)

    cd *= random.normalvariate(1.0, 0.03)

    return cl, max(cd, 0.0001)
```

```
def maybe_inject_anomaly(cl: float, cd: float, prob: float = 0.01) -> Tuple[float, float,
bool]:
```

```
    if random.random() >= prob:

        return cl, cd, False

    anomaly_type = random.choice(["negative_cd", "extreme_cl", "extreme_ld"])

    if anomaly_type == "negative_cd":

        return cl, -abs(cd), True

    if anomaly_type == "extreme_cl":
```

```

        return cl * 4.0, cd, True

    return cl, max(cd * 0.05, 1e-6), True

def build_dataset():

    airfoil_rows: List[AirfoilMeta] = []

    coordinate_rows: List[CoordinatePoint] = []

    performance_rows: List[PerformanceRecord] = []

    version_rows: List[Tuple[str, str, int, str]] = []

    dat_files = sorted(RAW_DIR.glob("*.dat"))

    if not dat_files:

        raise FileNotFoundError("raw_uiuc 目录下未找到 .dat 文件")

    selected_files = dat_files[:20]

    alpha_list = list(range(-4, 16, 2))

    re_list = [100000, 300000, 500000, 1000000]

    for file_path in selected_files:

        base_id = file_path.stem.lower().replace(" ", "_")

        base_name = file_path.stem

        base_coords = read_uiuc_dat(file_path)

        base_version_no = 1

        base_version_id = make_version_id(base_id, base_version_no)

        airfoil_rows.append(

            AirfoilMeta(

```

```

        airfoil_id=base_id,

        name=base_name,

        source="UIUC Airfoil Coordinates Database",

        family="unknown",

        is_generated=False,

    )

)

version_rows.append((base_version_id, base_id, base_version_no,
"imported_raw"))

for idx, (x, y) in enumerate(base_coords):

    coordinate_rows.append(

        CoordinatePoint(

            airfoil_id=base_id,

            version_id=base_version_id,

            point_order=idx + 1,

            x=x,

            y=y,

            surface=infer_surface(idx, len(base_coords)),

        )

    )

)

for re in re_list:

    for alpha in alpha_list:

        cl, cd = synthetic_aero_model(alpha, re)

        cl, cd, is_anomaly = maybe_inject_anomaly(cl, cd, prob=0.01)

        performance_rows.append(

```

```

        PerformanceRecord(
            airfoil_id=base_id,
            version_id=base_version_id,
            alpha_deg=alpha,
            reynolds_number=re,
            cl=cl,
            cd=cd,
            source_type="synthetic",
            is_anomaly=is_anomaly,
        )
    )

for k in range(2, 6):
    version_id = make_version_id(base_id, k)
    thickness_scale = random.normalvariate(1.0, 0.01)
    camber_shift = random.normalvariate(0.0, 0.002)
    new_coords = perturb_coordinates(base_coords, thickness_scale,
camber_shift)

    version_rows.append((version_id, base_id, k, "generated_variant"))

for idx, (x, y) in enumerate(new_coords):
    coordinate_rows.append(
        CoordinatePoint(
            airfoil_id=base_id,
            version_id=version_id,
            point_order=idx + 1,
            x=x,
            y=y,

```

```

        surface=infer_surface(idx, len(new_coords)),
    )
)

for re in re_list:

    for alpha in alpha_list:

        cl, cd = synthetic_aero_model(alpha, re)

        cl *= random.normalvariate(1.0, 0.015)

        cd *= random.normalvariate(1.0, 0.02)

        cl, cd, is_anomaly = maybe_inject_anomaly(cl, cd, prob=0.01)

        performance_rows.append(
            PerformanceRecord(
                airfoil_id=base_id,
                version_id=version_id,
                alpha_deg=alpha,
                reynolds_number=re,
                cl=cl,
                cd=cd,
                source_type="synthetic",
                is_anomaly=is_anomaly,
            )
        )

write_airfoils_csv(airfoil_rows)

write_coordinates_csv(coordinate_rows)

write_performance_csv(performance_rows)

```

```
write_versions_csv(version_rows)
```

```
print("数据生成完成。")
```

```
def write_airfoils_csv(rows: List[AirfoilMeta]):  
    with (OUT_DIR / "airfoils.csv").open("w", newline="", encoding="utf-8") as f:  
        writer = csv.writer(f)  
        writer.writerow(["airfoil_id", "name", "source", "family", "is_generated"])  
        for r in rows:  
            writer.writerow([r.airfoil_id, r.name, r.source, r.family, int(r.is_generated)])
```

```
def write_coordinates_csv(rows: List[CoordinatePoint]):  
    with (OUT_DIR / "coordinates.csv").open("w", newline="", encoding="utf-8") as f:  
        writer = csv.writer(f)  
        writer.writerow(["airfoil_id", "version_id", "point_order", "x", "y", "surface"])  
        for r in rows:  
            writer.writerow([r.airfoil_id, r.version_id, r.point_order, r.x, r.y, r.surface])
```

```
def write_performance_csv(rows: List[PerformanceRecord]):  
    with (OUT_DIR / "performance.csv").open("w", newline="", encoding="utf-8") as f:  
        writer = csv.writer(f)  
        writer.writerow([  
            "airfoil_id", "version_id", "alpha_deg", "reynolds_number",  
            "cl", "cd", "source_type", "is_anomaly"  
        ])  
        for r in rows:  
            writer.writerow([
```

```

        r.airfoil_id, r.version_id, r.alpha_deg, r.reynolds_number,

        r.cl, r.cd, r.source_type, int(r.is_anomaly)

    ])

def write_versions_csv(rows: List[Tuple[str, str, int, str]]):

    with (OUT_DIR / "data_versions.csv").open("w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        writer.writerow(["version_id", "airfoil_id", "version_no", "version_type"])

        for row in rows:

            writer.writerow(row)

if __name__ == "__main__":

    random.seed(2026)

    build_dataset()

```

4.7 生成数据脚本注意事项

该示例脚本是为了帮助同学们理解工程数据如何从真实来源进入数据库系统，并如何进一步形成适合数据库实验的数据集。同学们在使用该脚本或参考其思路时，必须满足以下要求：

- 第一，不能只提交生成后的 CSV 文件而不说明生成规则；
- 第二，必须区分真实导入数据与脚本生成数据，并在数据库中保留相应标记；
- 第三，必须说明版本数据、异常数据与原始数据之间的关系；
- 第四，不能将该脚本直接视为最终项目实现，仍需自行完成数据库建模、表结构设计、完整性约束、索引设计、查询实现和 AI 协同审计；
- 第五，若对性能生成模型进行了改动，必须说明改动依据，并说明其物理合理性和局限性。

4.8 说明与边界

本项目中的示例性能数据生成采用的是课程实验用简化模型，主要目的在于形成具有一定趋势性、可用于数据库实验的数据样本，而不是生成严格可信的航空工程仿真数据。因此，在项目报告中，学生必须说明：

哪些数据来自公开数据源；

哪些数据来自程序扩增或仿真生成；
生成数据所用规则是否具有物理合理性；
异常数据是自然存在还是为了教学实验故意注入。

五、系统功能要求

系统至少需要支持以下功能。

1. 翼型基础信息管理

能够记录翼型名称、编号、类别、来源、生成方式、备注信息等。

2. 几何坐标管理

能够管理翼型坐标点，支持按翼型读取完整轮廓，并区分上表面、下表面或其他必要标记。

3. 性能数据管理

能够记录不同攻角、雷诺数等条件下的性能数据，例如升力系数、阻力系数、升阻比等，并支持多条件查询。

4. 数据版本管理

当几何数据或性能数据修订后，系统应能保留版本信息，而不是直接覆盖原始记录。版本管理可以采用显式版本表，也可以采用带版本号的主表设计，但必须说明原因。

5. 异常数据识别

系统需实现至少一类规则检测，例如：
阻力系数为负；
升阻比异常偏离；
同一翼型在邻近攻角下性能变化不合理。

6. 查询与分析

系统应至少实现以下三类查询中的两类以上：

给定翼型编号，查询其全部几何坐标；
给定工况条件，查询满足性能阈值的翼型；
比较多个翼型在同一雷诺数下的性能曲线；
查询某翼型在不同版本下的性能差异；
识别含异常记录的翼型。

7. 数据导出与结果展示

至少支持将部分查询结果导出为 CSV 或图像，并能展示关键曲线分析结果。

六、数据库设计要求

1. 概念建模

每组必须提交完整的概念模型或 ER 图，并说明核心实体及其关系。

推荐至少包含以下实体：

Airfoil：翼型基本信息；

CoordinatePoint：翼型坐标点；

PerformanceRecord：性能数据；

DataVersion：版本信息；

DataSource：数据来源信息；

User：用户或操作人员信息。

如有需要，可扩展为：

ExperimentCondition、AnomalyRecord、QueryLog 等辅助实体。

2. 逻辑结构设计

每组必须说明：

主键与外键设计；

是否存在函数依赖；

是否达到第三范式，若未达到，为什么保留部分冗余；

哪些字段适合建立联合索引。

3. 约束要求

数据库中至少应包含以下几类约束：

主键约束；

外键约束；

非空约束；

唯一约束；

检查约束。

例如：

阻力系数 C_d 必须大于等于 0；

攻角 α 应在合理范围内；

同一版本下同一翼型同一工况不应重复插入。

七、数据库高级机制要求

1. 索引设计与性能实验

每组必须围绕至少一类核心查询进行索引优化实验。实验至少包括：

无索引情况下的查询耗时；
建立单列索引后的变化；
建立复合索引后的变化；
必要时删除某索引再对比。

必须提交：

查询语句；
执行计划截图或文本；
性能对比分析；
你们为什么选择该索引，而不是其他字段。

2. 事务或并发实验

每组至少完成一个事务相关场景。推荐选项如下：

两个用户同时修改同一条性能记录；
批量导入一组性能数据时，其中一条非法导致整批回滚；
更新翼型版本信息时，版本表与主表必须同时成功或同时失败。

3. 视图、触发器或存储过程

三者至少选择其一实现一个有实际意义的机制，例如：

通过视图统一展示“当前有效版本”；
通过触发器自动写入修改日志；
通过存储过程完成批量导入和合法性检查。

八、数据治理要求

本项目必须体现数据库对工程数据质量的治理作用。每组至少实现以下内容：

1. 异常规则检测

使用 SQL 或数据库逻辑检测异常记录，并将异常记录单独标识或写入异常表。

2. 版本追踪

能够说明某条数据来自哪个版本，何时写入，是否被修订。

3. 数据来源说明

至少区分真实来源数据与生成数据，并在数据库或文档中保留说明。

4. 删除策略说明

必须说明系统中哪些数据允许物理删除，哪些只能逻辑删除。
原则上，核心工程数据不建议直接删除。

九、数据智能协同模块

展示数据库系统如何与大模型可靠协作，每组必须完成以下两项，第三项作为优秀加分项。

必做一：自然语言到 SQL 的受控生成与审计

你们需要设计一组自然语言问题，使用大模型生成 SQL，并进行人工或程序审计。要求：

自然语言问题不少于 10 条；

至少覆盖单表查询、多表连接、分组统计、条件筛选中的若干类型；

逐条判断生成 SQL 是否正确；

总结错误类型。

重点分析以下问题：

是否出现字段名错误；

是否遗漏连接条件；

是否误用了聚合；

是否把版本表或条件表理解错；

加入 schema 提示后是否有改进。

必做二：查询结果的智能解释与审计

对于若干数据库查询结果，要求大模型给出解释，然后判断其是否合理。

分析重点包括：

解释是否真正基于结果；

是否符合基本工程常识；

是否出现“听起来合理但实际错误”的幻觉；

数据库结果不足时，大模型是否擅自补充不存在的信息。

选做三：异常检测对比实验

可以比较基于规则的检测与基于大模型的检测结果差异，讨论：

大模型是否真的提高了异常识别效果；

在哪些情况下规则方法更稳；

在哪些情况下大模型有辅助价值。

十、可视化分析要求

每组至少实现两项图形化分析，例如：

Cl 与 α 的关系曲线；

Cd 与 α 的关系曲线；

Cl/Cd 与 α 的关系曲线；
多个翼型在同一 Re 下的性能对比图；
不同版本翼型的性能差异图。

注意：

图形展示不是本项目的核心，但它可以帮助验证数据库是否真正支撑分析任务。

十一、科学思辨报告要求

每组必须在最终报告中单独设置“科学思辨与设计反思”部分，回答以下问题中的至少四个：

坐标点数据是否适合直接以 JSON 方式存储，为什么；
当数据规模扩大 100 倍后，当前模式和索引是否仍然适用；
性能数据是否适合列式数据库或分析型数据库；
版本表是独立设计更好，还是嵌入主表更好；
数据库与大模型协同的边界在哪里；
大模型是否适合直接参与事务写入；
异常检测中，规则方法与大模型方法各自适合什么场景。

严禁空泛议论，必须结合自己的系统设计作答。

十二、实现建议

建议使用 PostgreSQL 或 MySQL 完成数据库实现。

推荐 PostgreSQL，因为它在约束、执行计划分析、事务机制、视图、索引等方面更适合本项目训练目标。

应用层可以使用 Flask、FastAPI、Node.js 或命令行程序。

不要求复杂前端，不要求微服务，不要求云部署。

本项目的核心是数据库设计与协同分析能力，而不是前端界面包装。

十三、提交材料

每组需提交以下内容：

项目说明文档；
ER 图或概念模型图；
数据库建表脚本、约束脚本、索引脚本；
初始化数据或数据导入脚本；
系统源代码；
性能实验报告；
AI 协同测试日志；
科学思辨报告；

演示视频及现场答辩演示。

十四、评分标准

1. 概念建模与规范化分析（15 分）

是否正确抽象工程对象；
是否进行了基本规范化分析；
是否能解释自己的结构设计。

2. 表结构与完整性约束（15 分）

表设计是否合理；
是否存在明确主外键关系；
约束是否真正发挥作用。

3. 查询设计与索引优化（15 分）

是否围绕真实查询设计索引；
是否做了执行计划分析；
性能改进是否有依据。

4. 事务或一致性机制（10 分）

是否实现了至少一个有意义的事务场景；
是否理解数据库一致性问题。

5. 数据治理与版本管理（15 分）

是否能识别异常数据；
是否实现了版本追踪；
是否说明数据来源和删除策略。

6. 系统实现完整性（10 分）

系统是否能支撑核心流程；
是否形成完整可演示系统。

7. AI 协同审计能力（15 分）

是否真正测试并分析了 NL2SQL；
是否能指出大模型解释中的错误；
是否体现出批判性审计能力。

8. 科学思辨深度（5 分）

是否能对数据库与 AI 协同边界进行有根据的反思。

十五、加分项

能够区分“原始数据、派生数据、版本数据、异常数据”四类对象；

能够说明为什么某些字段必须单独建表而不能直接塞进 JSON；

能够用数据库机制而不是应用层硬编码保证数据规则；

能够证明索引设计与查询模式之间的关系；

能够对 AI 生成 SQL 的错误进行类型化总结；

能够明确指出大模型在工程数据解释中的边界，而不是一味肯定。